

Awesome 🚀 Let's move on to **Module 19: Transactions**.

SQL CLASS NOTE – MODULE 19: TRANSACTIONS

1. Introduction

A **Transaction** in SQL is a **sequence of operations** performed as a **single logical unit of work**.

👉 **Key rule:**

- A transaction must be **all or nothing**.
- If one part fails, the whole transaction fails and the database is rolled back to its previous state.

2. Properties of Transactions (ACID)

1. **Atomicity** – All steps succeed or none.
2. **Consistency** – Database moves from one valid state to another.
3. **Isolation** – Transactions don't interfere with each other.
4. **Durability** – Once committed, data is permanently saved.

3. Syntax

```
START TRANSACTION;  -- or BEGIN;
    -- SQL operations here
COMMIT;             -- Save changes
ROLLBACK;           -- Undo changes
```

4. Practical Examples with SchoolDB

Example 1: Insert student and enrollment as one transaction

```
START TRANSACTION;
```

```
INSERT INTO Students (StudentID, FirstName, LastName, Age, Gender, GradeLevel)  
VALUES (201, 'Michael', 'Scott', 16, 'Male', 'Grade 11');
```

```
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)  
VALUES (501, 201, 105);
```

```
COMMIT;
```

👉 If the enrollment insert fails, we can `ROLLBACK` and the student will not be added either.

Example 2: Rollback when something goes wrong

```
START TRANSACTION;
```

```
UPDATE Students  
SET Age = Age + 1  
WHERE GradeLevel = 'Grade 12';
```

```
-- Suppose we realize it was a mistake:
```

```
ROLLBACK;
```

👉 The changes are undone.

Example 3: Commit after multiple updates

```
START TRANSACTION;
```

```
UPDATE Students SET Age = Age + 1 WHERE StudentID = 101;
```

```
UPDATE Students SET Age = Age + 1 WHERE StudentID = 102;
```

```
COMMIT;
```

👉 Both updates are applied together.

Example 4: Delete student and enrollment together

```
START TRANSACTION;
```

```
DELETE FROM Enrollments WHERE StudentID = 110;
```

```
DELETE FROM Students WHERE StudentID = 110;
```

```
COMMIT;
```

👉 Ensures no orphaned enrollments remain.

5. Savepoints

👉 A **Savepoint** allows partial rollback within a transaction.

```
START TRANSACTION;
```

```
UPDATE Students SET Age = 20 WHERE StudentID = 105;
```

```
SAVEPOINT sp1;
```

```
UPDATE Students SET Age = 25 WHERE StudentID = 106;
```

```
ROLLBACK TO sp1;    -- Undo only changes after sp1
```

```
COMMIT;
```

6. Why Use Transactions?

-  Ensure **data integrity**.
-  Group multiple changes into one unit.
-  Prevent partial updates in case of errors.

7. Real-Life Use Cases

- **Banks**: Transfer money (debit + credit must both succeed).
- **E-commerce**: Order placement (reduce stock + record payment).
- **Schools**: Student enrollment (insert student + assign course).
- **Hospitals**: Patient admission (insert record + allocate bed).

8. Classwork (Practice)

1. Start a transaction to add a new student and enroll them in 2 courses. Commit if successful.
2. Start a transaction to update all Grade 10 students' age by +1. Rollback if mistake.
3. Start a transaction to delete a student and all their enrollments.
4. Use a **Savepoint** inside a transaction to partially rollback.
5. Demonstrate a failed transaction where one insert works and the other fails → rollback everything.

9. Challenge Question

 Simulate a **bank transfer** using the SchoolDB:

- Deduct 5 credits from one student's account.
- Add 5 credits to another student's account.
- Ensure both succeed or none (use transaction with commit/rollback).

 That's the **expanded Module 19: Transactions**.

Perfect  Let's solve the **Module 19 Classwork + Challenge** step by step with SQL queries.



Solutions to Module 19: TRANSACTIONS

1. Start a transaction to add a new student and enroll them in 2 courses. Commit if successful.

```
START TRANSACTION;
```

```
INSERT INTO Students (StudentID, FirstName, LastName, Age, Gender, GradeLevel)
VALUES (202, 'Linda', 'Brown', 15, 'Female', 'Grade 10');
```

```
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)
VALUES (601, 202, 101);
```

```
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)
VALUES (602, 202, 102);
```

```
COMMIT;
```

2. Start a transaction to update all Grade 10 students' age by +1. Rollback if mistake.

```
START TRANSACTION;
```

```
UPDATE Students
SET Age = Age + 1
WHERE GradeLevel = 'Grade 10';
```

```
-- Suppose we realize it was wrong
ROLLBACK;
```

3. Start a transaction to delete a student and all their enrollments.

```
START TRANSACTION;

DELETE FROM Enrollments WHERE StudentID = 108;
DELETE FROM Students WHERE StudentID = 108;

COMMIT;
```

4. Use a **Savepoint** inside a transaction to partially rollback.

```
START TRANSACTION;

UPDATE Students SET Age = 18 WHERE StudentID = 105;
SAVEPOINT sp1;

UPDATE Students SET Age = 19 WHERE StudentID = 106;

-- Rollback only second update
ROLLBACK TO sp1;

COMMIT;
```

5. Demonstrate a failed transaction where one insert works and the other fails → rollback everything.

```
START TRANSACTION;

INSERT INTO Students (StudentID, FirstName, LastName, Age, Gender, GradeLevel)
VALUES (203, 'Kevin', 'White', 14, 'Male', 'Grade 9');

-- This will fail if CourseID = 999 does not exist
INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID)
VALUES (603, 203, 999);

-- Since one insert fails, rollback
ROLLBACK;
```



Challenge Question

👉 Simulate a **bank transfer** using the SchoolDB:

(Assume *Students* table has a *Credits* column)

- Deduct 5 credits from Student 101.
- Add 5 credits to Student 102.
- Ensure both succeed or none.

```
START TRANSACTION;
```

```
UPDATE Students
```

```
SET Credits = Credits - 5
```

```
WHERE StudentID = 101;
```

```
UPDATE Students
```

```
SET Credits = Credits + 5
```

```
WHERE StudentID = 102;
```

```
-- If both succeed
```

```
COMMIT;
```

```
-- If error occurs
```

```
-- ROLLBACK;
```

✅ This ensures **data consistency**: either both updates happen, or none happen.